

把技术笔记变成一份好读的 PDF

这是一份 Markdown 导出样稿。内容包含中文正文、英文术语、列表、表格、常用公式和一个跨页代码块，重点观察日常技术笔记的阅读体验。

01 / 正文与信息层次

写笔记时，我们希望注意力停留在内容上。导出以后，标题应该容易定位，段落之间留有呼吸空间，代码也应该保留清晰的缩进。**可复制的正文**与稳定的分页，比把整个网页截成一张长图更实用。

一份好的导出文档，不需要把编辑器的按钮、网址和日期一起带走。

- 标题与正文保持一致的对齐关系。
- 中文、英文和数字混排，例如 Vue 3、UTF-8 与 2026。
- 代码沿用等宽字体，长内容可以连续跨页。

内容类型	这份样稿的处理
正文与列表	真实 PDF 文字，可选择与搜索
长代码块	文本分页，保留缩进与行号
独立公式	MathJax 生成矢量图形
页眉与页脚	不添加标题、网址、日期或页码

02 / 日常公式

先看一个常见的加权平均。它包含求和、上下标和分式，用于检查公式的清晰度与段落间距。

$$\bar{x}_w = \frac{\sum_{i=1}^n w_i x_i}{\sum_{i=1}^n w_i}$$

正态分布的概率密度包含指数和根号；这里保留独立公式的自然尺寸，不用位图截图。

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

03 / 一个连续跨页的代码块

下面是一个完整的本地笔记检索示例。它对标题命中赋予更高权重，并为正文命中设置上限。这段代码不手动插入分页符，可以检查从一页到下一页时是否漏行、错序或截断。

```
1  // A small search utility for a local Markdown notebook.
2  // All filtering stays local; source notes are never modified.
3
4  const defaults = {
5    limit: 20,
6    minScore: 0,
7    caseSensitive: false,
8  };
9
10 function normalize(value, caseSensitive = false) {
11   const text = String(value ?? '').trim();
12   return caseSensitive ? text : text.toLocaleLowerCase();
13 }
14
15 function tokenize(query) {
16   return [...new Set(query.split(/\s+/).filter(Boolean))];
17 }
18
19 function countMatches(text, token) {
20   let count = 0;
21   let offset = 0;
22
23   while (offset < text.length) {
24     const index = text.indexOf(token, offset);
25     if (index === -1) break;
26     count += 1;
27     offset = index + token.length;
28   }
29
30   return count;
31 }
32
33 function scoreNote(note, tokens, options) {
34   const title = normalize(note.title, options.caseSensitive);
35   const body = normalize(note.body, options.caseSensitive);
36   let score = 0;
37
38   for (const token of tokens) {
39     const titleHits = countMatches(title, token);
```

```

40     const bodyHits = countMatches(body, token);
41     score += titleHits * 3 + Math.min(bodyHits, 10);
42 }
43
44 return score;
45 }
46
47 function makeSnippet(body, token, radius = 48) {
48     const text = String(body ?? '');
49     const index = text.toLocaleLowerCase().indexOf(token);
50     const start = Math.max(0, index - radius);
51     const end = Math.min(text.length, start + radius * 2);
52     const prefix = start > 0 ? '...' : '';
53     const suffix = end < text.length ? '...' : '';
54     return prefix + text.slice(start, end) + suffix;
55 }
56
57 export function searchNotes(notes, query, overrides = {}) {
58     const options = { ...defaults, ...overrides };
59     const normalized = normalize(query, options.caseSensitive);
60     const tokens = tokenize(normalized);
61     if (tokens.length === 0) return [];
62
63     const matches = [];
64     for (const note of notes) {
65         const score = scoreNote(note, tokens, options);
66         if (score <= options.minScore) continue;
67
68         matches.push({
69             id: note.id,
70             title: note.title,
71             score,
72             snippet: makeSnippet(note.body, tokens[0]),
73         });
74     }
75
76     matches.sort((left, right) => {
77         const scoreDifference = right.score - left.score;
78         if (scoreDifference !== 0) return scoreDifference;
79         return String(left.id).localeCompare(String(right.id));
80     });
81
82     return matches.slice(0, options.limit);
83 }

```

```
84
85 // A caller can use the same function for live previews.
86 const notes = [
87   { id: 'a', title: 'Markdown export', body: 'Keep text searchable.' },
88   { id: 'b', title: 'Code pagination', body: 'Allow code to cross pages.' },
89 ];
90
91 const results = searchNotes(notes, 'code', { limit: 5 });
92 console.log(JSON.stringify(results, null, 2));
93 // END_OF_CODE
```

代码后的正文应该紧接着出现。看到这一段，说明文档没有把后续内容遗留在代码容器之外。

04 / 矩阵与多行推导

向量和矩阵也是技术笔记中的常见内容。下面用二维线性变换检验括号大小和列对齐。

$$\begin{pmatrix} y_1 \\ y_2 \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

再放一组简短推导，检查等号对齐与多行公式的垂直间距。

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - \theta x_i)^2$$
$$\frac{\partial \mathcal{L}}{\partial \theta} = -\frac{2}{n} \sum_{i=1}^n x_i (y_i - \theta x_i)$$

05 / 导出后的检查

- 中文是否清楚，正文是否能够选中复制。
- 长代码是否连续，末尾的 END_OF_CODE 是否完整保留。
- 公式放大后是否清晰，矩阵和等号是否对齐。
- 每页是否保留留白，是否没有多余的页眉页脚。

这份样稿使用独立的文档样式。它演示的是 pdfmake 排版与 MathJax 公式的组合效果，尚未接入编辑器的导出按钮。

全文结束。